음성 AI 전문가 과정 · Day 27 / 40

Day 27

Pipecat 기초
Frame · Processor · Pipeline · Transport

```
Pipeline([
  transport.input(),
  VADProcessor(...), WhisperSTT(),
  user_agg, make_llm(), KoTTS(),
  transport.output(),
  assistant_agg,
])

# 말 끝 → 첫 소리 ≈ 1 ~ 2초
```

오늘 완성하는 파이프라인 모양

Frame 세 종류 → Processor 규칙 → Worker · 방향 · Observer → STT · TTS · LLM 서비스 → 조립 (두 턴 대화) → WebSocket Transport
6 주차 M6 · 4 시간 · pipecat-ai 1.15.0 · 키 없으면 로컬 Qwen3

0.0 · 오늘 답할 질문

비서를 함수 줄 세우기에서 " 공장 " 으로

1 소리 조각 · 글자 · 신호는 모두 어떤 모양으로 흐르나 ? Part 1

2 작업자 (Processor) 를 만들 때 꼭 지킬 규칙은 ? Part 2

3 파이프라인은 누가 돌리고 , 알림은 어느 방향으로 가나 ? Part 3

4 우리 Whisper · TTS · Qwen 을 어떻게 서비스로 감싸나 ? Part 4

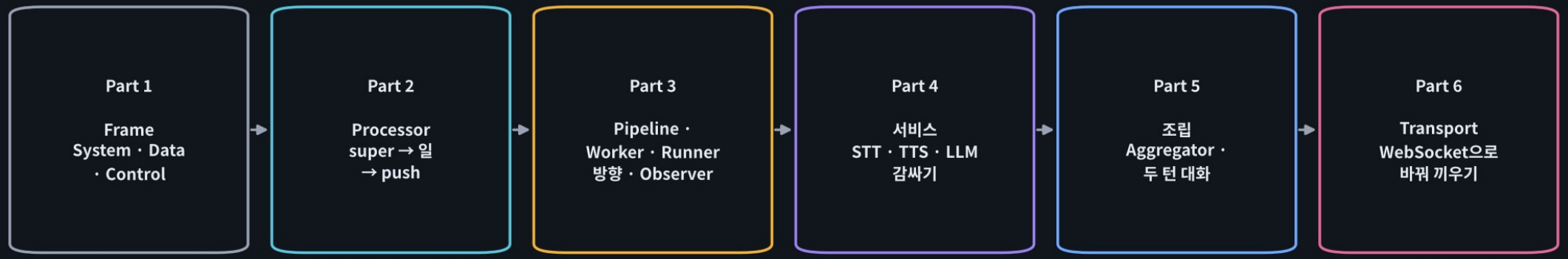
5 턴 끝 판단과 대화 기록은 누가 맡나 ? Part 5

6 같은 비서를 WebSocket(나중엔 브라우저 · 전화) 에 붙이려면 ? Part 6

0.0 · 오늘의 흐름

상자 · 작업자 · 라인 · 서비스 · 조립 · 문

D27 · 상자(Frame)를 작업자(Processor)들이 줄지어 넘기는 공장: Pipecat으로 STT → LLM → TTS 잇기



D26의 프레임 · 스트리밍 · Transport 개념이 Pipecat의 틀 안으로 → D28 끼어들기 · 턴 판단 · Metrics의 바탕

교시	내용	빈칸	애니메이션
1 교시	Part 0 ~ 2 설치 · Frame · FrameProcessor	1 · 2 · 3	D27-1 · 2 · 3
2 교시	Part 3 Pipeline · Worker · Runner · 방향 · Observer	4	D27-4
3 교시	Part 4 서비스 STT · TTS · 루프 막힘 · LLM	5 · 6 · 7	D27-5 · 6
4 교시	Part 5 ~ 7 조립 · Transport · 정리	8 · 9	D27-7 · 8 · 9

함수 줄 세우기로는 실시간이 안 된다

```
# D25:  차례로 부르는 함수
text  = stt(record_until_silence())
reply = llm(messages + [text])
audio = tts(reply)          #  다 끝나야 말함
play(audio)                 #  그동안 귀는 닫힘
```

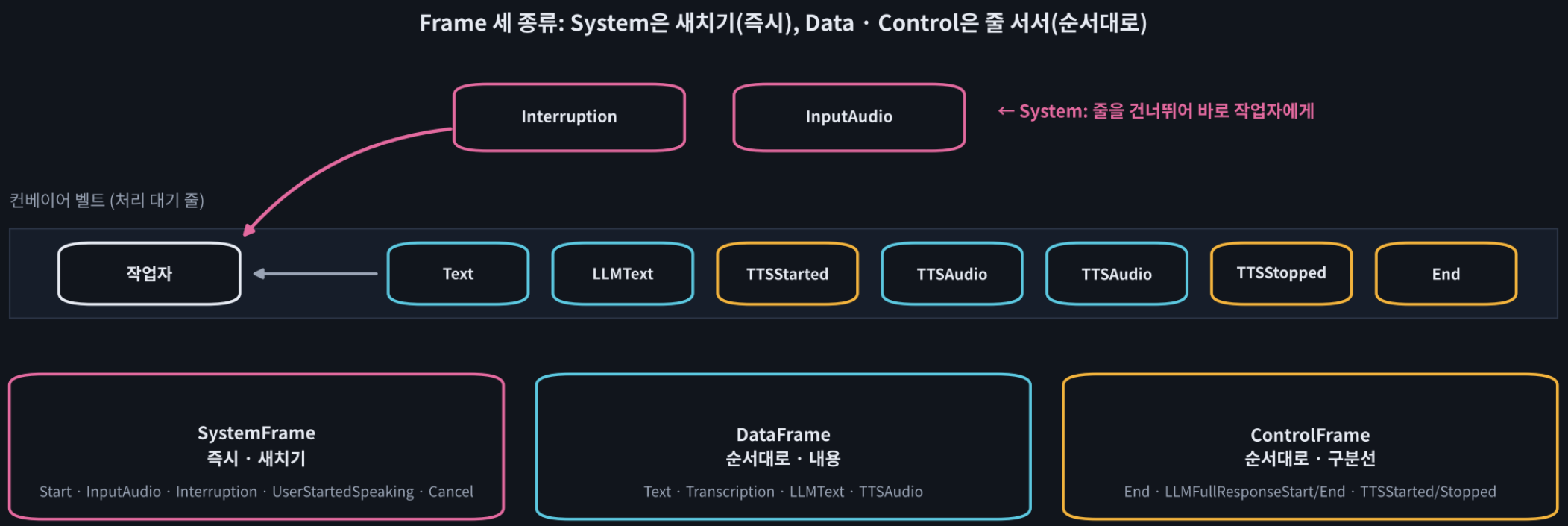
```
# D27:  상자를 흘려보내는 공장
InputAudio → VAD → STT → user_agg
  → LLM (LLMText  조각… ) → TTS ( 문장마다 )
  → output → assistant_agg
#  듣는 중에도 ,  말하는 중에도 계속 흐름
```

🍎 **컨베이어 벨트 공장**

상자 (Frame) 가 벨트 위로 흐르고 , 작업자 (Processor) 는 자기 상자만 열어 일하고 나머지는 그대로 넘긴다 . 화재 경보 (SystemFrame) 는 줄을 기다리지 않는다 .

💬 **쉽게 말하면** 모든 것을 Frame 에 담고 , Processor 들이 받아 → 일하고 → 넘긴다 . 이 규칙 하나로 실시간 흐름을 짠다 .

System 은 즉시 · Data · Control 은 순서대로



▶ 애니메이션 D27-2 Frame 세 종류

📐 **System = 즉시 , Data · Control = 순서대로**

Frame 분류기

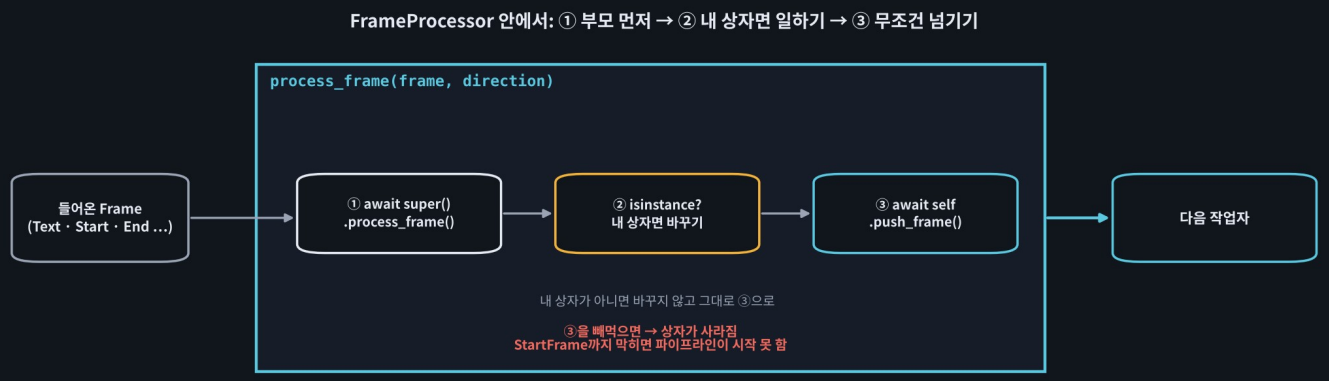
```
def kind(f):
    if isinstance(f, SystemFrame):
        return "System"
    if isinstance(f, ____):      # 1
        return "Data"
    return "Control"
```

🧩 빈칸 1

내용 (글자 · 소리) 을 담는 Frame 의 부모 클래스 이름은 ?

Frame	종류
StartFrame · InputAudioRawFrame	System
VADUserStartedSpeakingFrame	System
InterruptionFrame	System
TranscriptionFrame · LLMTextFrame	Data
TTSAudioRawFrame	Data
LLMFullResponseStartFrame	Control
EndFrame	Control

부모 먼저 → 내 일 → 무조건 넘기기




```python
class KoNormalizer(FrameProcessor):
  async def process_frame(self, frame, direction):
    await super().____(frame, direction)  # 2
    if type(frame) is TextFrame:
      frame = TextFrame(text=normalize_ko(...))
    await self.____(frame, direction)     # 3
```

🍎 릴레이 바통

바통을 받으면 내 구간만 달리고 반드시 다음 주자에게 . 주머니에 넣으면 (push 빼먹기) 경기가 멈춘다 .

▶ 애니메이션 D27-3 Processor 안에서

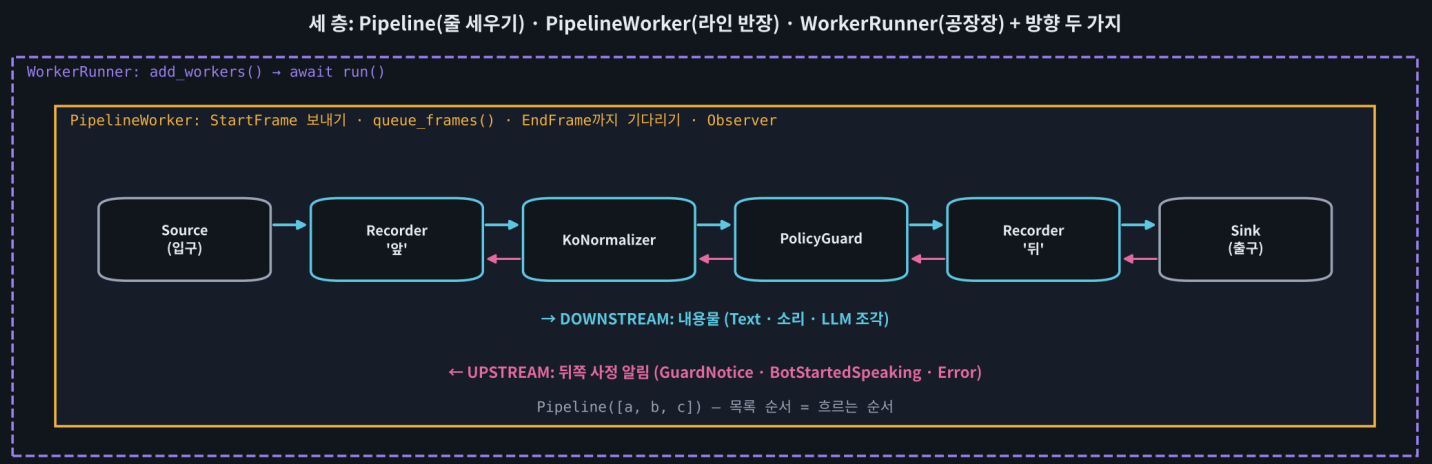
push_frame 을 빼먹으면 StartFrame 이 막힌다

```
class Forgetful(FrameProcessor):
  async def process_frame(self, frame, direction):
    await super().process_frame(frame, direction)
    if isinstance(frame, TextFrame):
      await self.push_frame(...)
    # else:  빼먹음 ← 흔한 실수
```

```
ERROR | PipelineWorker#1: timeout waiting
  for StartFrame#2 to reach the end of the
  pipeline (being blocked somewhere?),
  stopping the pipeline.
'뒤'가 받은 Frame 수: 0
```

1. Worker 가 StartFrame 을 맨 앞에서 보낸다
2. StartFrame 이 맨 끝 (Sink) 에 닿아야 "준비 완료"
3. 중간에서 사라지면 기다리다 시간 초과 (기본 20 초, 실험은 3 초)

라인 배치도 · 라인 반장 · 공장장



층	하는 일
Pipeline	목록 순서로 줄 세움
PipelineWorker	Start · 상자 넣기 · 끝 처리
WorkerRunner	여러 Worker 실행 · 정리

```
worker = PipelineWorker(pipeline, ...)
runner = WorkerRunner(handle_sigint=False)
await runner.add_workers(worker)
await runner.run()
```

⚠ 인터넷 예제의 PipelineTask · PipelineRunner = 옛 이름 (2.0 에서 제거 예정)

알림은 뒤에서 앞으로 (UPSTREAM)

```
@dataclass
class GuardNoticeFrame(SystemFrame):
    reason: str = ""
...
if "주민번호" in frame.text:
    await self.push_frame(notice, FrameDirection.____)  # 4
    return   #  원래 상자는 의도적으로 버림
```

```
class FlowLog(BaseObserver):
  async def on_push_frame(self, data):
    data.source → data.destination
    data.frame · data.direction
    data.timestamp · data.first_push
PipelineWorker(..., observers=[flow])
```

▶ 애니메이션 D27-4 방향

💬 **쉽게 말하면** 내용물은 앞 → 뒤 , " 뒤쪽 사정 " 알림은 뒤 → 앞으로 흐른다 .

서비스 = FrameProcessor + 모델 호출 하나



기본 제공 서비스(Deepgram · OpenAI · Anthropic · Cartesia · ElevenLabs …)로 Pipeline 목록의 한 칸만 바꿔 끼울 수 있다

▶ 애니메이션 D27-5 서비스 세 개

📐 **run_stt · run_tts 는 yield 로 Frame 을 내보낸다**

Whisper → STT 서비스 , tts() → TTS 서비스

```
class WhisperSTT(SegmentedSTTService):
  @property
  def wants_wav_segments(self): return False
  async def run_stt(self, audio: bytes):
    y = np.frombuffer(audio, np.int16) / 32768
    text = await asyncio.to_thread(stt, y)
    yield ____(text=text, ...)   # 5
```

```
class KoTTS(TTSService):
  async def run_tts(self, text, context_id):
    y = await asyncio.____(tts,          # 6
                normalize_ko(text))
    yield TTSAudioRawFrame(audio=pcm,
        sample_rate=SR, num_channels=1,
        context_id=context_id)
```

서비스	받음	냄
WhisperSTT	VAD 말 시작 ~ 끝 구간 PCM	TranscriptionFrame
KoTTS	문장 (LLM 조각을 모은 것)	TTSStarted · TTSAudioRawFrame … · TTSStopped

무거운 호출은 asyncio.to_thread 로

방식	박동 최대 간격	늦은 박동
그냥 호출 tts() · stt()	수백 ~ 1,000+ ms	있음
await asyncio.to_thread(...)	20 ~ 30 ms	0

출력 모양 예시 · 실제 값은 모델 · GPU 마다 다름

▶ 애니메이션 D27-6 이벤트 루프 막힘

1. 20 ms 마다 마이크 조각이 와야 한다 (D26)
2. 루프가 600 ms 막히면 조각 30 개가 쌓이고 VAD · 끼어들기 감지가 멈춘다
3. PyTorch 는 계산 중 GIL 을 풀어 주므로 스레드로 충분

🍎 계산대 하나뿐인 가게

점원이 창고에 직접 물건을 가지러 가면 (그냥 호출) 계산대 줄이 멈춘다 . 창고 담당 (스레드) 에게 맡기면 계산은 계속 .

LLMContextFrame 을 받아 LLMTextFrame 조각으로

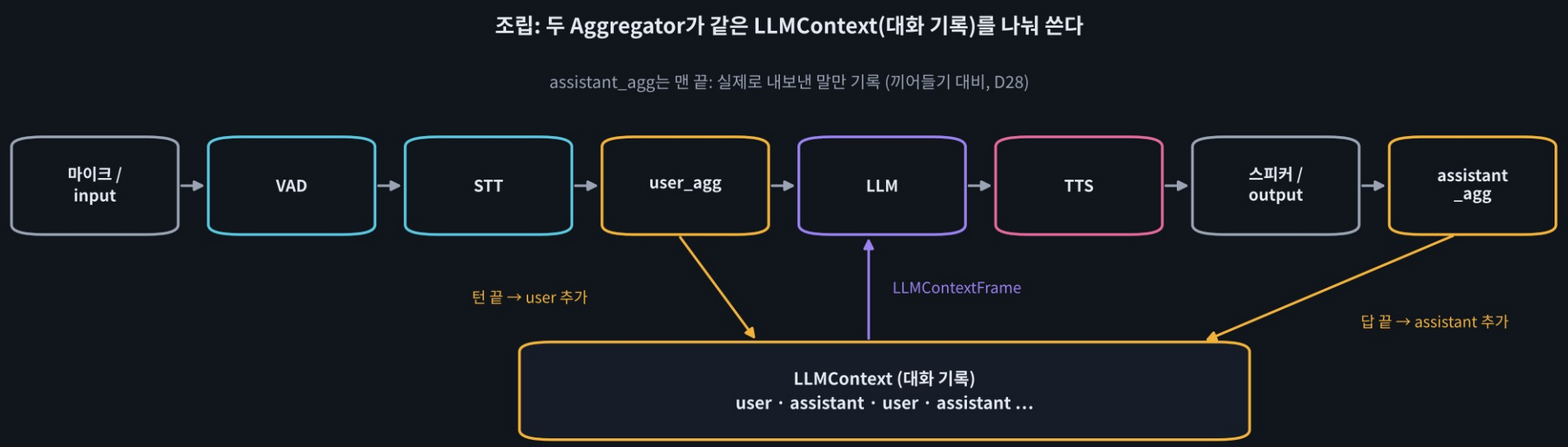
```
class LocalLLM(FrameProcessor):
  async def process_frame(self, frame, direction):
    await super().process_frame(frame, direction)
    if isinstance(frame, ____):        # 7
      push(LLMFullResponseStartFrame())
      for piece in qwen: push(LLMTextFrame(piece))
      push(LLMFullResponseEndFrame())
```

PROVIDER	base_url
openai	(기본)
claude	api.anthropic.com/v1/
gemini	…googleapis.com/v1beta/openai/
local	LocalLLM(qwen_stream)

```
OpenAILLMService(api_key=secret(KEYS[P]), base_url=BASE_URLS[P],
    settings=OpenAILLMService.Settings(model=MODELS[P], system_instruction=SYSTEM))
```

시스템 프롬프트는 1.9 부터 LLM 서비스의 system_instruction 에 (LLMContext 의 system 메시지는 옛 방식)

두 Aggregator 가 같은 대화 기록을 나눠 쓴다



1. VAD 말 끝 → STT 받아쓰기
2. 0.6 초 조용 + 받아쓰기 도착 → 턴 끝 → LLM
3. LLM 조각 → TTS 문장 → 소리 → 기록

▶ 애니메이션 D27-7 조립된 한 턴

두 턴 대화 : 비서가 첫 턴을 기억한다

```
context, user_agg, assistant_agg = make_bot()     # LLMContext() + LLMContextAggregatorPair
parts = [VADProcessor(...), WhisperSTT(), user_agg, make_llm(), KoTTS(), speaker, ____]   # 8
await run_frames(parts, feed=mic(VOICE1, VOICE2, wait=(speaker, LLMFullResponseEndFrame)))
```

role	content (출력 모양 예시)
user	안녕하세요 한빛치과죠 다음 주 수요일 오후 두 시에 스케일링 예약하고 싶어요
assistant	네 , 다음 주 수요일 오후 2 시 스케일링 말씀이시죠 ? 예약하실 분 성함을 알려 주세요 .
user	네 이름은 김민준이에요
assistant	감사합니다 , 김민준 님 . 다음 주 수요일 오후 2 시로 예약해 드렸습니다 .

💬 **쉽게 말하면** D22 에서 손으로 하던 messages.append 를 두 Aggregator 가 대신한다 .

말 끝 → 첫 소리는 어디서 쓰이나

```
응답 ≈ max( 턴 끝 대기 0.6 초 , STT) + LLM 첫 글자 + 첫 문장 모으기 + TTS 첫 문장
```

턴	받아쓰기	턴 끝 판정	LLM 첫 글자	첫 소리
1	0.4	0.6	1.1	1.4
2	0.3	0.6	1.0	1.3

출력 모양 예시 (T4 · 클라우드 LLM 가정 , 초) · 실행마다 다름

🍎 대화 중 숨 고르기

" 수요일… 오후 두 시에 " 사이의 0.4 초 침묵을 끝으로 보면 말을 자른다 . 길면 매 턴 어색한 침묵 .

1. TURN_WAIT 0.3 · 0.6 · 1.0 으로 바꿔 두 턴 다시 (과제 3)
2. GPU 에서는 " 조용함 기다리기 " 가 , CPU 에서는 STT 가 큰 덩어리
3. D28: Smart Turn · 끼어들기 · Metrics

문만 바꾸고 안쪽은 그대로



Transport	바깥
SingleClientWebsocketServer	WebSocket (오늘)
FastAPIWebsocket + Twilio	전화망
SmallWebRTC	브라우저
Daily · LiveKit	여러 사람 방

▶ 애니메이션 D27-8 Transport 바꿔 끼우기

💬 **쉽게 말하면** 바깥의 약속 (PCM · μ-law · WebRTC) 이 바뀌면 Transport 와 Serializer 만 갈아 끼운다 .

Serializer: 바이트 ⇄ Frame

```
class RawPCMSerializer(FrameSerializer):
  async def serialize(self, frame):
    if isinstance(frame, ____):     # 9
      return frame.audio
    return None
  async def deserialize(self, data):
    return InputAudioRawFrame(audio=data, ...)
```

WebSocket 턴	말 끝 → 첫 소리	답 길이
1	≈ 1.4 초	≈ 3.8 초
2	≈ 1.5 초	≈ 3.5 초

출력 모양 예시 · 같은 컴퓨터 안 · 실행마다 다름

1. 클라이언트 : D26 처럼 20 ms 조각을 실시간 속도로 보냄
2. 서버 : transport.input() → … → transport.output() 이 소리를 실시간 속도로 돌려줌
3. 손님이 끊으면 on_client_disconnected → EndFrame → 정상 종료

모든 것은 Frame, 작업자는 받고 · 일하고 · 넘긴다

개념	코드
Frame	`isinstance(f, DataFrame)`
Processor	`super().process_frame · self.push_frame`
방향	`FrameDirection.UPSTREAM`
STT	`yield TranscriptionFrame(...)`
TTS	`await asyncio.to_thread(tts, ...)`
LLM	`LLMContextFrame → LLMTextFrame…`
조립	`..., output, assistant_agg]`
Transport	`serialize: OutputAudioRawFrame`

다음 : D28 Pipecat 심화

오늘은 "0.6 초 조용하면 끝 " 으로 턴을 끊었고 끼어들기는 보지 않았다 . D28: Smart Turn · Interruption(Barge-in) · VAD 튜닝 · Metrics 로 지연 측정 .

`worksheet.html` 40 문항

개념 워크시트 · 노션 제출

`animation.html` 9 개 탭

인터랙티브 애니메이션

`D27_Pipecat 기초 _ 실습 .ipynb` 빈칸 9

실습 노트북 · 키 선택

온라인 자료 imguru-mooc.github.io/AI_Voice/day27